Egyptian Academy for Engineering & Advanced Technology

Mechatronics Engineering Department

“Simple oscilloscope”

Course Name

Microcontrollers - ECO 335

Signals and Systems - ECO 363

Submitted by

Abdelrahman Mohamed Maher – 2022118
Ahmed Mohamed Ahmed – 2022128

Supervisor

Dr. Amin Danial
Dr. Yasser El Shrief

Dec 2025

Table of Contents

Table of Contents i

Table of Figures ii

1. Abstract 1

2. Components 2

3. Arduino uno 3

3.1 Arduino uno pin diagram 3

3.2 Arduino uno pin description 3

3.2.2 Analog Pins (A0–A5) 4

4. MAX7219 4-in-1 8x8 5

4.1 Purpose 5

5. Analog-to-Digital Converter (ADC) 6

5.1 ADC overview 6

5.2 Operation Modes 6

5.3 Use of ADC in the Project 7

6. Hardware simulation and circuit 8

7. Code Explanation 9

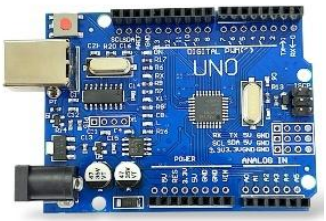
Table of Figures

Figure 3.1 Arduino Pin Diagram 3
Figure 4.1 MAX7219 4-in-1 8x8 connection 5
Figure 5.1 ADC overview 6
Figure 7.1 Simulation Circuit 8
Figure 7.2 Hardware Circuit 8

1. Abstract

This project implements a simple digital oscilloscope using an AVR microcontroller and a 4-in-1 MAX7219 8×8 LED matrix display. The microcontroller samples an analog input signal via its ADC, stores the data, and visualizes the waveform in real-time on the LED matrix. Timer interrupts manage ADC sampling and display refreshing, while a pause button allows the waveform to be temporarily frozen. The MAX7219 simplifies driving multiple LED matrices, enabling clear and efficient waveform representation.

2. Components

Name	Number	Image
Arduino uno	1	
MAX7219 4-in-1 8x8 module	1	
Push button	2	

3. Arduino uno

3.1 Arduino uno pin diagram

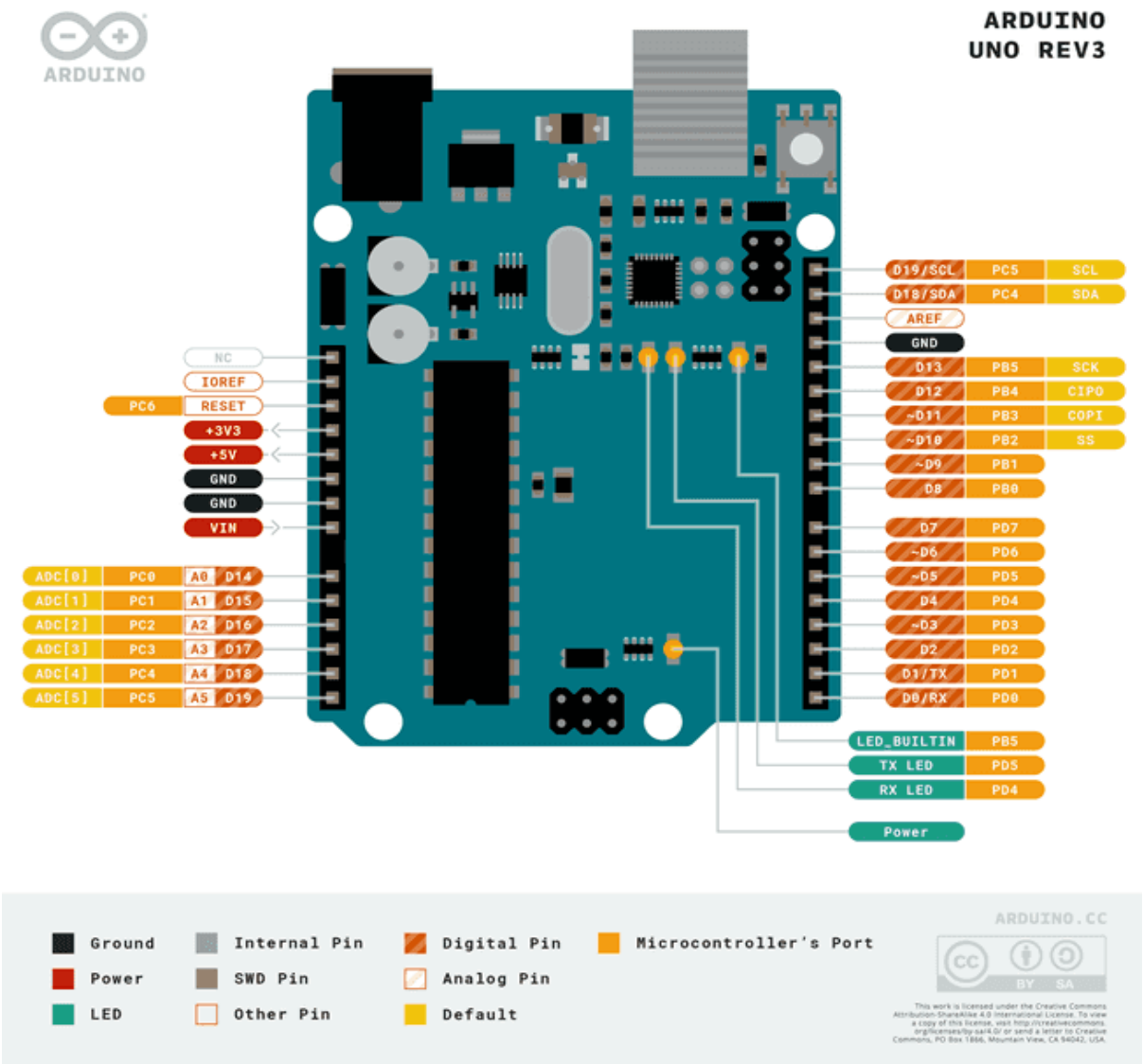


Figure 3.1 Arduino Pin Diagram

3.2 Arduino uno pin description

3.2.1 Digital Pins (0–13)

- Function: Can be used as input or output.
- Voltage: 5V logic levels.
- Current: Max 20 mA per pin, 40 mA absolute maximum.
- Special pins:
- Pins 0 (RX) and 1 (TX): Used for serial communication (UART). Be careful using them while uploading code.
- Pins 3, 5, 6, 9, 10, 11: Can do PWM (Pulse Width Modulation) output using analog Write ().

3.2.2 Analog Pins (A0–A5)

- **Function:** Primarily used for **analog input** (0–5V) to the ADC (10-bit resolution: 0–1023).
- **Can also be used as digital pins** (use digital Read()/digital Write()).

3.2.3 Power Pins

- **Vin:** Input voltage to Arduino when using an external power supply (7–12V recommended).
- **5V:** Regulated 5V output from the board (can power other circuits).
- **3.3V:** 3.3V output (limited current, ~50 mA).
- **GND:** Ground.
- **RESET:** Can be used to reset the Arduino manually.

3.2.4 ICSP Header (In-Circuit Serial Programming)

- Used for programming the **ATmega328P** directly.
- Contains **MISO, MOSI, SCK, RESET, VCC, GND**.

3.2.5 Communication Pins

- **Serial (UART):** Pins 0 (RX), 1 (TX)
- **I2C:** A4 (SDA), A5 (SCL)
- **SPI:** 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK)

3.2.6 LED

- Built-in LED is connected to **digital pin 13**. Can use LED_BUILTIN constant.

3.2.7 Reset Button

- Pressing it resets the microcontroller, starting the program from the beginning.

4. MAX7219 4-in-1 8x8

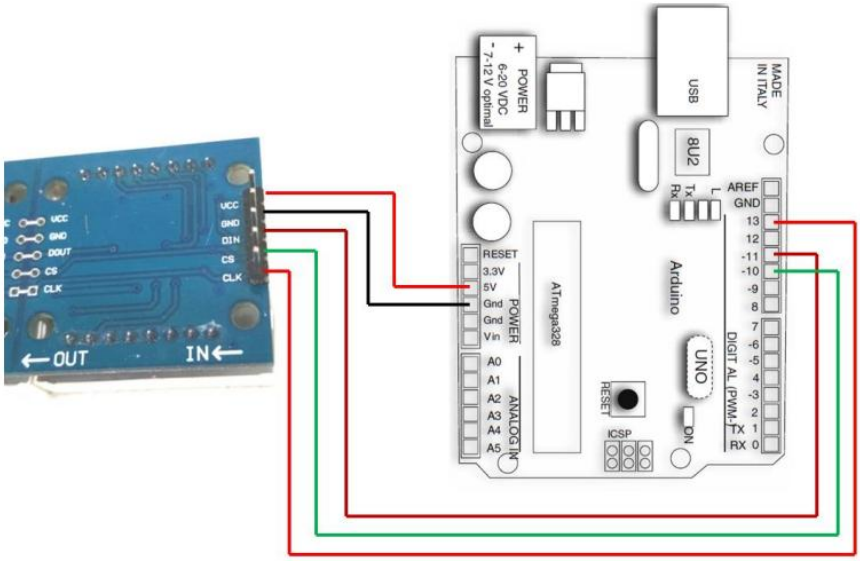


Figure 4.1 MAX7219 4-in-1 8x8 connection

4.1 Purpose

❖ **Function:**

- Controls multiple LED matrices, converting MCU digital data into LED patterns with brightness control.

❖ **Structure:**

- **4 units chained → 32×8 LEDs.**
- **Key pins: DIN, DOUT, CLK, CS/LOAD, VCC, GND.**
- Internal registers handle row scanning and LED states.

❖ **How it works:**

- MCU sends serial data → first unit → cascaded through DOUT.
- Each row updated sequentially; IC handles multiplexing.

❖ **Usage in the project:**

- Displays ADC waveform as vertical bars.
- Timer interrupts update the display smoothly.
- Pause button freezes the display without affecting data collection.

5. Analog-to-Digital Converter (ADC)

5.1 ADC overview

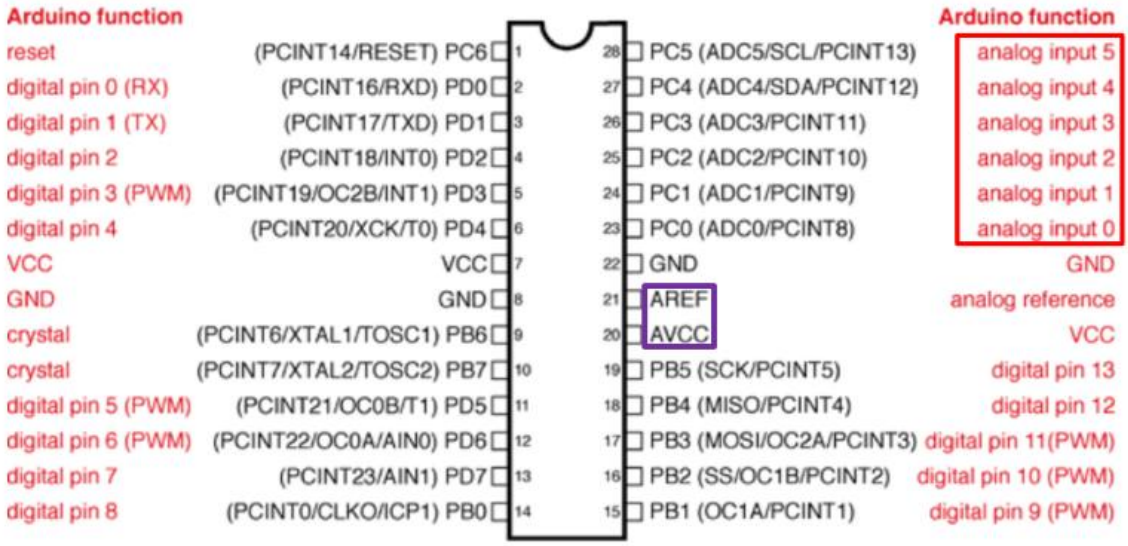


Figure 5.1 ADC overview

5.2 Operation Modes

- **Single Conversion Mode:**
  - The ADC performs a single conversion when triggered and then stops. You need to start each conversion manually by setting the ADSC bit in the ADCSRA register.
  - **Usage**: Suitable for applications where occasional ADC readings are sufficient
- **Free Running Mode:**
  - The ADC continuously performs conversions as soon as the previous conversion completes. This mode is enabled by setting the ADATE bit in the ADCSRA register.
  - **Usage**: Useful for applications requiring constant monitoring of an analog signal.
- **Analog Comparator Mode:**
  - The ADC conversion is triggered by an analog comparator event. This mode requires configuring the analog comparator and setting the appropriate trigger source.
  - **Usage**: Suitable for applications needing precise event-driven ADC conversions.

- **External Interrupt Trigger Mode:**
  - The ADC conversion is triggered by an external interrupt. This mode can be useful for synchronizing ADC conversions with external events.
  - **Usage**: Ideal for scenarios where ADC conversions need to be synchronized with external hardware signals.

- **Timer/Counter Trigger Modes:**
  - The ADC conversion is triggered by specific timer/counter events, such as compare match or overflow. This mode is set by configuring the ADC Auto Trigger Source (ADTS) bits in the ADCSRB register.
  - **Usage**: Useful for precise timing applications, like sampling at regular intervals.

5.3 Use of ADC in the Project

The **ADC (Analog-to-Digital Converter)** of the AVR microcontroller is used to **sample the input Analog waveform**. It converts the continuous voltage signal into discrete digital values that the microcontroller can process. In this project:

- **Sampling frequency** is controlled by Timer1 (1 kHz).
- Each ADC value is **scaled to 0–7** to match the 8 rows of the LED matrix.
- The collected samples are stored in an array of 32 points.
- These digital values are then used to **build the LED display**, showing the waveform in real-time.
- The ADC allows visualizing analog signals on the LED matrix without external converters.

6. Hardware simulation and circuit

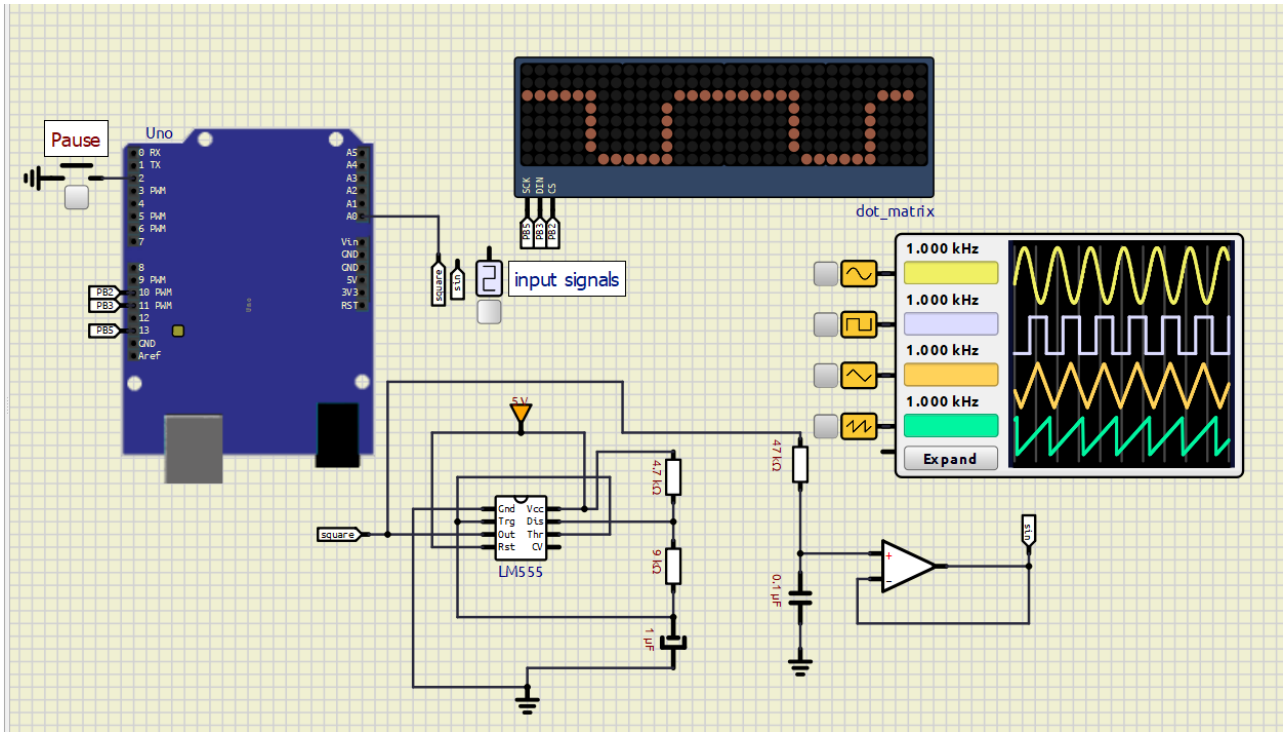


Figure 6.1 Simulation Circuit

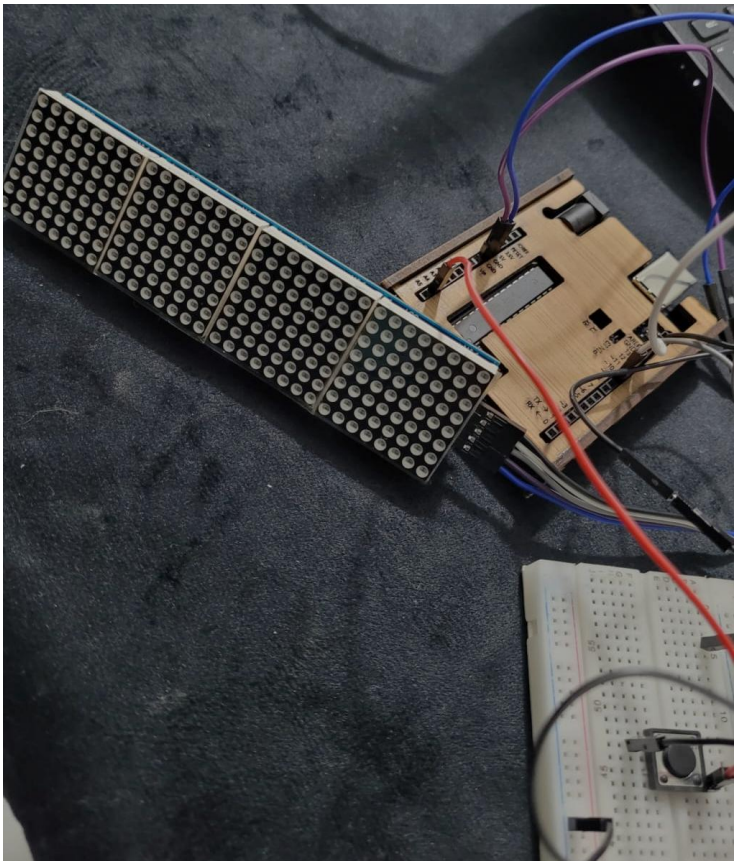


Figure 6.2 Hardware Circuit

7. Code Explanation

```c
#include <avr/io.h>
#include <avr/interrupt.h>

//CONFIG
const unsigned char NUMBER_OF_MODULES = 4;
const unsigned char NUMBER_OF_SAMPLES = 32;

//GLOBAL VARIABLES
volatile unsigned char current_sample_position = 0;
volatile unsigned char sample_array[32];
volatile unsigned char buffer_is_ready = 0;
volatile unsigned char pause_acquisition = 0;

unsigned char led_frame[8][4];

//SPI FUNCTIONS
void spi_send_byte(unsigned char one_byte)
{
    SPDR = one_byte;
    while ((SPSR & 0x80) == 0);
}

void max7219_send(unsigned char reg, unsigned char data)
{
    PORTB &= ~(1 << 2); //CS LOW

    for (unsigned char i = 0; i < NUMBER_OF_MODULES; i++)
    {
        spi_send_byte(reg);
        spi_send_byte(data);
    }

    PORTB |= (1 << 2); //CS HIGH
}
```

```c
//MAX7219 INIT
void max7219_initialize(void)
{
    max7219_send(0x0C, 0x01);
    max7219_send(0x09, 0x00);
    max7219_send(0x0B, 0x07);
    max7219_send(0x0A, 0x08);
    max7219_send(0x0F, 0x00);
}

//ADC INIT
void adc_initialize(void)
{
    ADMUX  = 0b01000000; // AVcc reference, ADC0
    ADCSRA = 0b10001110; // enable ADC, interrupt, prescaler 64
}

//TIMER1 INIT
void timer1_initialize(void)
{
    TCCR1B = 0b00001000; // CTC mode
    OCR1A  = 249;        // 1 kHz sampling
    TCCR1B |= 0b00000011; // prescaler 64
    TIMSK1 = 0b00000010; // enable compare interrupt
}
```

```
//INTERRUPTS
ISR(TIMER1_COMPA_vect)
{
    ADCSRA |= (1 << 6); //start ADC conversion
}

ISR(ADC_vect)
{
    if (pause_acquisition) return;

    unsigned int adc_value = ADC;
    sample_array[current_sample_position] = (adc_value * 7) / 1023;
    current_sample_position++;

    if (current_sample_position >= NUMBER_OF_SAMPLES)
    {
        current_sample_position = 0;
        buffer_is_ready = 1;
    }
}
```

```c
//BUILD FRAME
void build_frame_from_samples(const unsigned char source_samples[])
{
    for (unsigned char row = 0; row < 8; row++)
        for (unsigned char module = 0; module < NUMBER_OF_MODULES; module++)
            led_frame[row][module] = 0;

    for (unsigned char column = 0; column < NUMBER_OF_SAMPLES - 1; column++)
    {
        unsigned char row1 = 7 - source_samples[column];
        unsigned char row2 = 7 - source_samples[column + 1];
        unsigned char module_number = column / 8;
        unsigned char bit_position = 7 - (column % 8);

        led_frame[row1][module_number] |= (1 << bit_position);

        unsigned char difference = (row1 > row2) ? (row1 - row2) : (row2 - row1);
        if (difference >= 2)
        {
            signed char step = (row2 > row1) ? 1 : -1;
            for (signed char y = row1; y != row2; y += step)
                led_frame[y][module_number] |= (1 << bit_position);
        }
    }
}
```

```c
//REFRESH DISPLAY
void refresh_display(void)
{
    for (unsigned char row = 0; row < 8; row++)
    {
        PORTB &= ~(1 << 2); // CS LOW
        for (unsigned char module = 0; module < NUMBER_OF_MODULES; module++)
        {
            spi_send_byte(row + 1);
            spi_send_byte(led_frame[row][module]);
        }
        PORTB |= (1 << 2); // CS HIGH
    }
}
```

```c
int main(void)
{
    // SPI pins
    DDRB |= (1 << 2) | (1 << 3) | (1 << 5);
    SPCR = 0b01010001;

    //Pause button on PD2 (input with pull-up)
    DDRD &= ~(1 << 2);
    PORTD |= (1 << 2);

    max7219_initialize();
    adc_initialize();
    timer1_initialize();
    sei();

    unsigned char snapshot[NUMBER_OF_SAMPLES];
    unsigned int refresh_counter = 0;

    //Debounce variables
    unsigned char last_button_state = 1;
    unsigned char stable_button_state = 1;
    unsigned char debounce_counter = 0;
```

```
    while (1)
    {

        unsigned char current_button_state = (PIND & (1 << 2)) ? 1 : 0;

        if (current_button_state != stable_button_state){
            debounce_counter++;
            if (debounce_counter >= 5)
            {
                stable_button_state = current_button_state;
                if (stable_button_state == 0)
                {
                    pause_acquisition = !pause_acquisition;
                }
                debounce_counter = 0;
            }
        }
        else
        {
            debounce_counter = 0;
        }

        //UPDATE DISPLAY
        if (buffer_is_ready)
        {
            cli();
            for (unsigned char i = 0; i < NUMBER_OF_SAMPLES; i++)
                snapshot[i] = sample_array[i];
            buffer_is_ready = 0;
            sei();

            build_frame_from_samples(snapshot);
        }
        // refresh display ~200 Hz
        refresh_counter++;
        if (refresh_counter >= 5)
        {
            refresh_display();
            refresh_counter = 0;
        }
    }
}

```